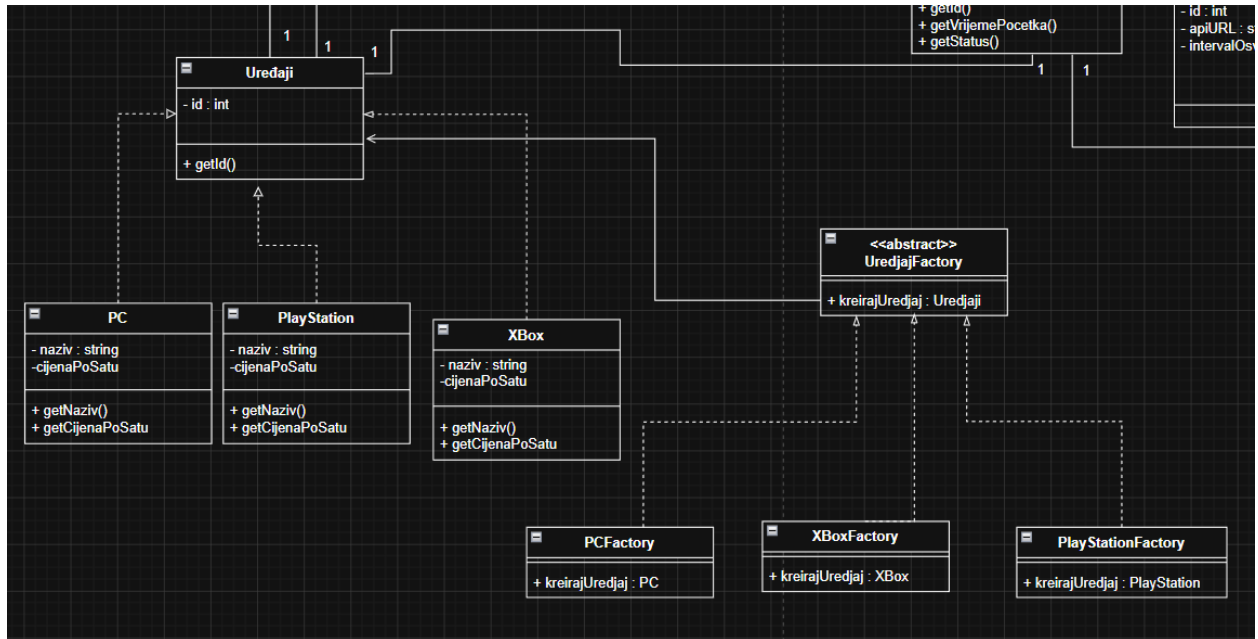


KREACIJSKI PATERNI

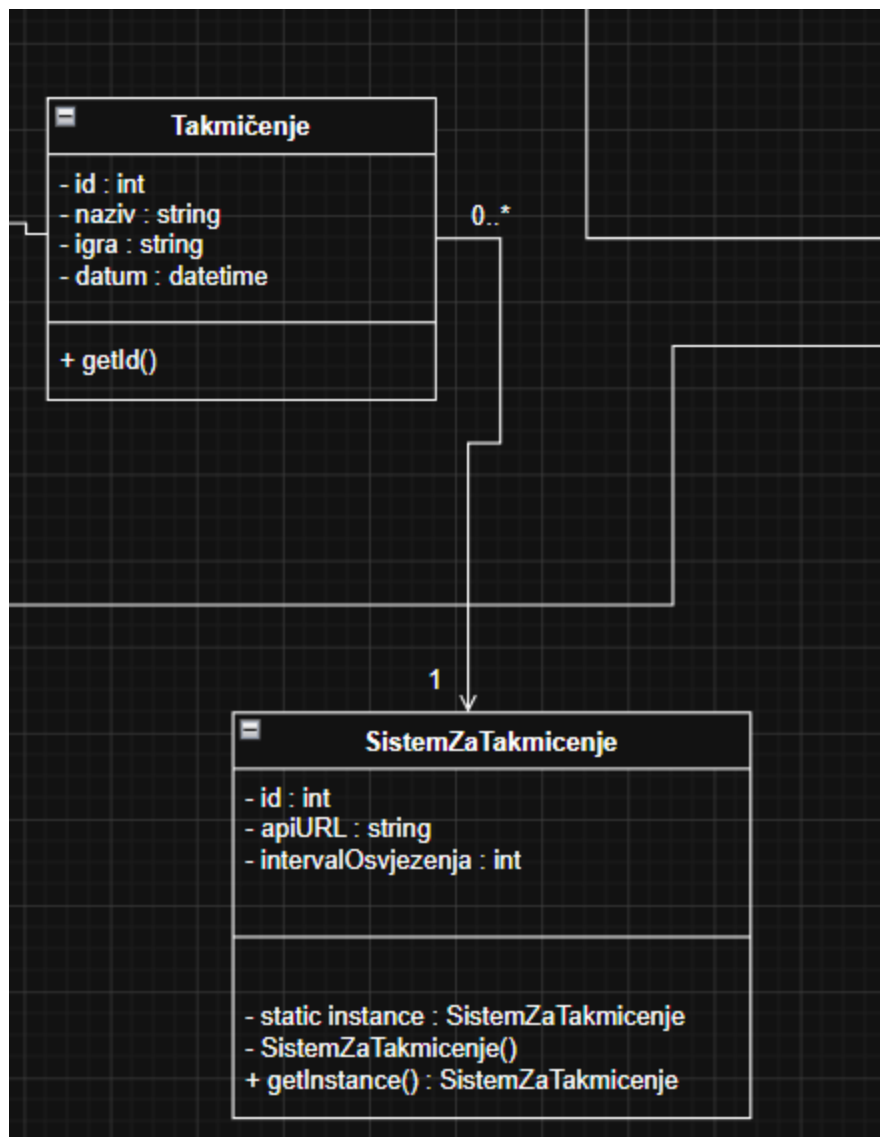
FACTORY METHOD PATTERN

Factory method pattern odvaja zahtijevanje objekata od kreiranja objekata. U našem sistemu najbolje ga je uključiti kod klase *Uredjaji*. Kreiranje objekata se delegira posebnoj fabrici umjesto direktnog pozivanja konstruktora. Nas trenutni sistem ima: PC, PlayStation i Xbox. Dodajemo apstraktnu fabriku *UredjajFactory* i konkretne fabrike *PCFactory*, *PlayStationFactory* i *XBoyFactory*.



SINGLETON PATTERN

Singleton pattern osigurava da u sistemu postoji samo jedna instanca određene klase i omogućava globalni pristup toj instanci. Najbolji kandidat u našem sistemu sa ovaj pattern je *SistemZaTakmicenje*. Gaming centar treba imati jedan centralni sistem za pregled takmičenja (ukoliko dodje do nadogradnje evidenciju turnira), jedinstvenu konekciju prema API-ju i centralno osvježanje podataka. Tako bismo trosili manje resursa i lakše osvježavali podatke.



ABSTRACT FACTORY PATTERN

Kreira zone povezanih objekata. U sistemu bi se mogao koristiti za kompletne gaming konfiguracije. Npr. PC zona: PC, PC periferija, PC cijene; Console zona: PlayStation/Xbox, kontroleri, TV. Pogodno za koristiti ukoliko sistem postane previše veliki i sastoji se od više gaming zona i različite konfiguracije i opreme.

BUILDER PATTERN

Postepeno kreiranje kompleksnog objekta. U sistemu se Builder pattern može koristiti za kreiranje rezervacije. Rezervacija može imati: uređaj, termin, dodatne usluge, loyalty popust, više igrača i posebne zahtjeve. Pogodnosti: preglednije kreiranje i fleksibilnost.

Primjer:

RezervacijaBuilder

.setVrijeme()

.setUredjaj()

.setPopust()

.build();

PROTOTYPE PATTERN

Kopiranje postojećeg objekta umjesto kreiranja od početka. Često se dešavaju ista takmičenja, tako da je moguće klonirati postojeće takmičenje i promijeniti datum. Npr. Takmicenje novo = staroTakmicenje.clone();.